

Designing the architecture of the RAG feature pipeline

The last step is to architect and go through the design of the RAG feature pipeline of the LLM Twin application. We will use a batch design scheduled to poll data from the MongoDB data warehouse, process it, and load it to a Qdrant vector DB. The first question to ask ourselves is, “Why a batch pipeline?”

But before answering that, let’s quickly understand how a batch architecture works and behaves relative to a streaming design.

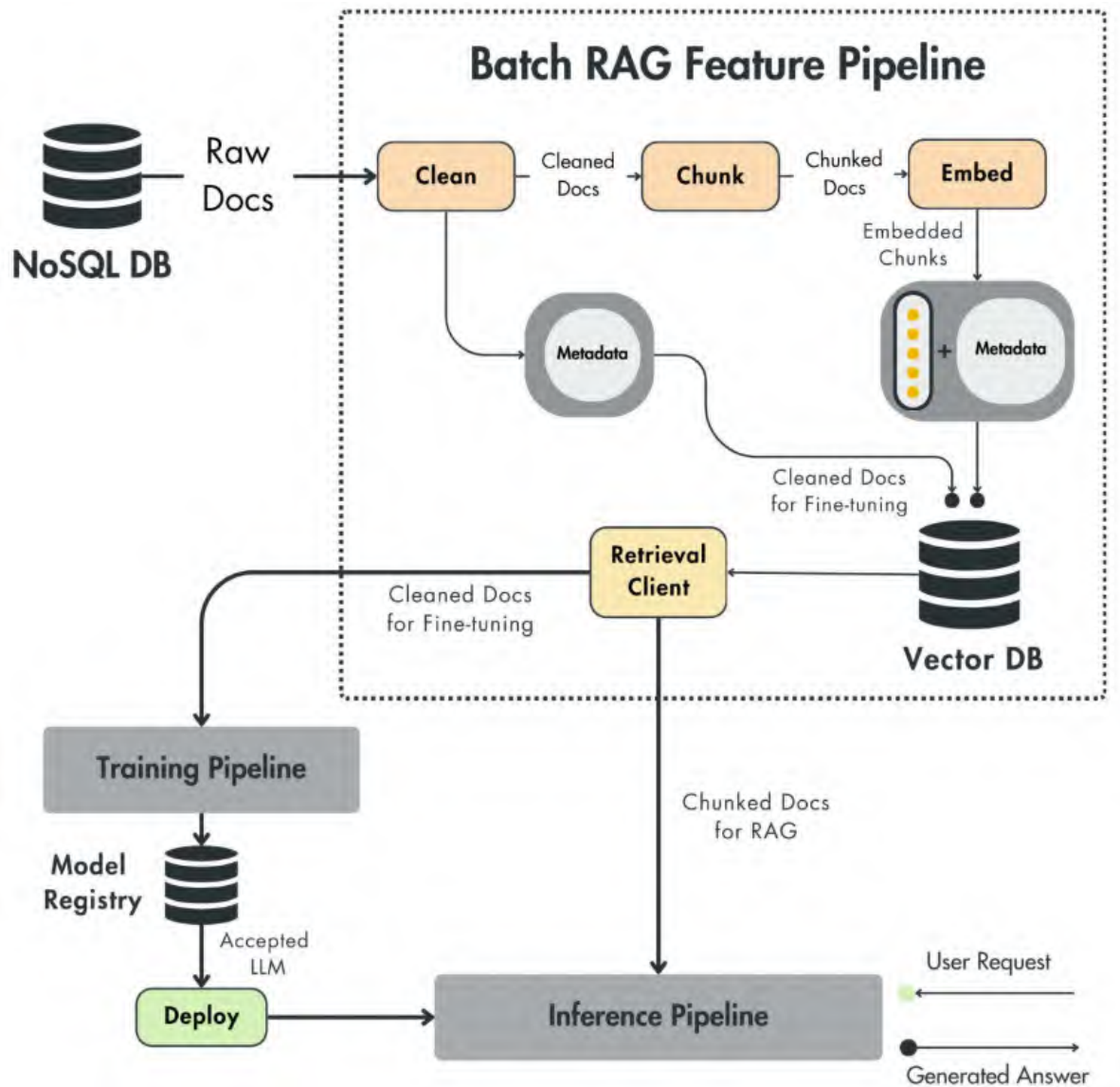


Figure 4.9: The architecture of the LLM Twin's RAG feature pipeline

Batch pipelines

A batch pipeline in data systems refers to a data processing method where data is collected, processed, and stored in predefined intervals and larger volumes, also known as “batches”. This approach differs from real-time or streaming data processing, where data is processed continuously as it arrives. This is what happens in a batch pipeline:

1. **Data collection:** Data is collected from various sources and stored until sufficient amounts are accumulated for processing. This can include data from DBs, logs, files, and other sources.
2. **Scheduled processing:** Data processing is scheduled at regular intervals, for example, hourly or daily. During this time, the collected data is processed in bulk. This can involve data cleansing, transformation, aggregation, and other operations.
3. **Data loading:** After processing, the data is loaded into the target system, such as a DB, data warehouse, data lake, or feature store. This processed data is then available for analysis, querying, or further processing.

Batch pipelines are particularly useful when dealing with large volumes of data that do not require immediate processing. They offer several advantages, including:

- **Efficiency:** Batch processing can handle large volumes of data more efficiently than real-time processing, allowing for optimized resource allocation and parallel processing.
- **Complex processing:** Batch pipelines can perform complex data transformations and aggregations that might be too resource-intensive for real-time processing.
- **Simplicity:** Batch processing systems’ architectures are often simpler than those of real-time systems, making them easier to implement and maintain.

Batch versus streaming pipelines

When implementing feature pipelines, you have two main design choices: batch and streaming. Thus, it is worthwhile to see the difference between the two and understand why we chose a batch architecture over a streaming one for our LLM Twin use case.

You can effortlessly write a dedicated chapter on streaming pipelines, which suggests its complexity over a batch design. However, as streaming architectures become increasingly popular, one must have an intuition of how they work to choose the best option for your application.

The core elements of streaming applications are a distributed event streaming platform such as Apache Kafka or Redpanda to store events from multiple clients and a streaming engine such as Apache Flink or Bytewax to process the events. To simplify your architecture, you can swap your event streaming platform with queues, such as RabbitMQ, to store the events until processed. *Table 4.1* compares batch and streaming pipelines based on multiple criteria such as processing schedule and complexity:

Aspect	Batch pipeline	Streaming pipeline
Processing schedule	Processes data at regular intervals (e.g., every minute, hourly, daily).	Processes data continuously, with minimal latency.
Efficiency	Handles large volumes of data more efficiently, optimizing resource allocation and parallel processing.	Handles single data points, providing immediate insights and updates, allowing for rapid response to changes.
Processing complexity	Capable of performing complex data transformations and aggregations.	Designed to handle high-velocity data streams with low latency.
Use cases	Suitable for scenarios where immediate data processing is not critical. Commonly used in data warehousing, reporting, ETL processes, and feature pipelines.	Ideal for applications requiring real-time analytics, features, monitoring, and event-driven architectures.
System complexity	Compared to streaming pipelines, systems are generally simpler to implement and maintain.	More complex to implement and maintain due to the need for low-latency processing, fault tolerance, and scalability. The tooling is also more advanced and complicated.

Table 4.1: Batch versus streaming pipelines

For example, streaming pipelines are extremely powerful in social media recommender systems like TikTok. When using social media, user behavior changes frequently. A typical scenario is that you want to relax at a certain point in time and mostly look at videos of puppies. Still, after 15 minutes, you get bored and want something more serious, such as educative content or news. This means the recommender system has to capture these behavior changes without delay to keep you engaged. As the transition between interests is cyclical and not predictable, you can't use a batch pipeline that runs every 30 minutes or every hour to generate more content. You can run it every minute to create new content, but, at the same time, it will result in unnecessary costs, as most predictions will not be consumed. By implementing a streaming pipeline, you update the features of specific users in real time, which are then passed to a chain of models that predict the new recommendations.

Streaming architectures are also the backbone of real-time fraud detection algorithms, such as those used at Stripe or PayPal. In this context, it's critical to identify potentially fraudulent transactions as they occur, not after a few minutes or hours as a batch pipeline would process them. The same urgency applies to high-frequency trading platforms that make stock predictions based on the constant influx of market data, enabling traders to make decisions within milliseconds.

On the other hand, you can use a batch architecture for an offline recommender system. For example, when implementing one for an e-commerce or streaming platform, you don't need the system to be so reactive, as the user's behavior rarely changes. Thus, updating the recommendations periodically, such as every night, based on historical user behavior data using a batch pipeline is easier to implement and cheaper.

Another popular example of batch pipelines is the ETL design used to extract, transform, and load data for different use cases. The ETL design is widespread in data pipelines used to move data from one DB to another. Some practical use cases include aggregating data for analytics, where you have to extract data from multiple sources, aggregate it, and load it to a data warehouse connected to a dashboard. The analytics domains can be widespread, from e-commerce and marketing to finance and research.

The data collection pipeline used in the LLM Twin use case is another example of an ETL pipeline that extracts data from the internet, structures it, and loads it into a data warehouse for future processing.

Along with prediction or feature freshness, another disadvantage of batch pipelines over streaming ones is that you usually make redundant predictions. Let's take the example of a recommender system for a streaming platform like Netflix. Every night, you make the predictions for all users. There is a significant chance that a large chunk of users won't log in that day. Also, users usually don't browse all the recommendations but stick to the first ones. Thus, only a portion of predictions are used, wasting computing power on all the others.

That's why a popular strategy is to start with a batch architecture, as it's faster and easier to implement. After the product is in place, you gradually move to a streaming design to reduce costs and improve the user experience.

To conclude, we have used a batch architecture (and not a streaming one) to implement the LLM Twin's feature pipeline for the following reasons:

- **Does not require immediate data processing:** Even if syncing the data warehouse and feature store is critical for an accurate RAG system, a delay of a few minutes is acceptable. Thus, we can schedule the batch pipeline to run every minute, constantly syncing the two data storages. This technique works because the data volume is small. The whole data warehouse will have only thousands of records, not millions or billions. Hence, we can quickly iterate through them and sync the two DBs.
- **Simplicity:** As stated earlier, implementing a streaming pipeline is two times more complex. In the real world, you want to keep your system as simple as possible, making it easier to understand, debug, and maintain. Also, simplicity usually translates to lower infrastructure and development costs.

In *Figure 8.10*, we compare what tools you can use based on your architecture (streaming versus batch) and the quantity of data you have to process (small versus big data). In our use case, we are in the smaller data and batch quadrant, where we picked a combination of vanilla Python and generative AI tools such as LangChain, Sentence Transformers, and Unstructured.

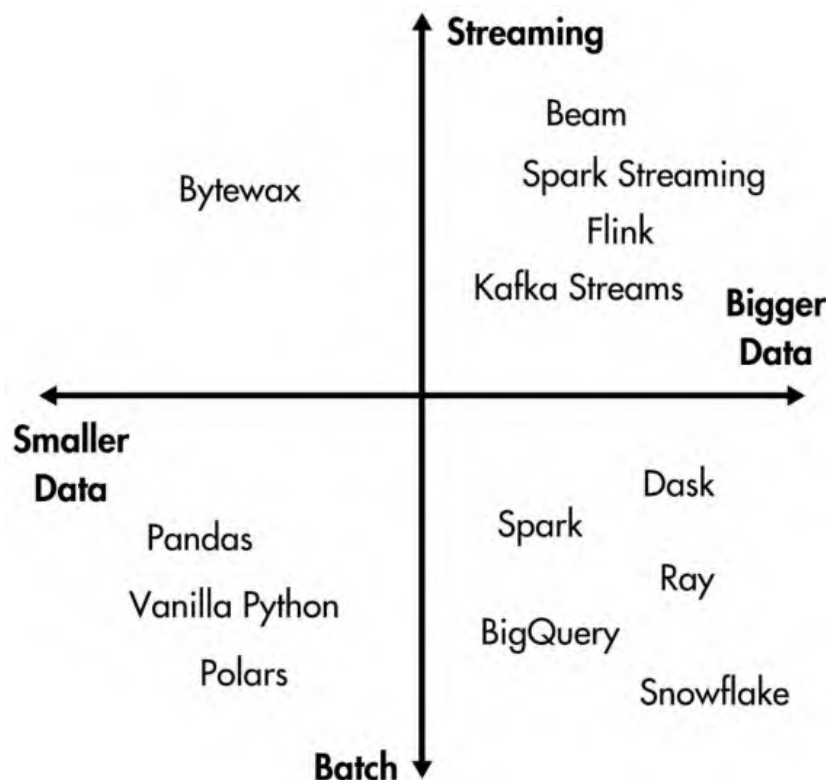


Figure 4.10: Tools on the streaming versus batch and smaller versus bigger data spectrum

In the *Change data capture: syncing the data warehouse and feature store* section later in this chapter, we will discuss when switching from a batch architecture to a streaming one makes sense.

Core steps

Most of the RAG feature pipelines are composed of five core steps. The one implemented in the LLM Twin architecture makes no exception. Thus, you can quickly adapt this pattern for other RAG applications, but here is what the LLM Twin's RAG feature pipeline looks like:

1. **Data extraction:** Extract the latest articles, code repositories, and posts from the MongoDB data warehouse. At the extraction step, you usually aggregate all the data you need for processing.

2. **Cleaning:** The data from the data warehouse is standardized and partially clean, but we have to ensure that the text contains only useful information, is not duplicated, and can be interpreted by the embedding model. For example, we must clean and normalize all non-ASCII characters before passing the text to the embedding model. Also, to keep the information semantically dense, we decided to replace all the URLs with placeholders and remove all emojis. The cleaning step is more art than **science**. Hence, after you have the first iteration with an evaluation mechanism in place, you will probably reiterate and improve it.
3. **Chunking:** You must adopt various chunking strategies based on each data category and embedding model. For example, when working with code repositories, you want the chunks broader, whereas when working with articles, you want them narrower or scoped at the paragraph level. Depending on your data, you must decide if you split your document based on the chapter, section, paragraph, sentence, or just a fixed window size. Also, you have to ensure that the chunk size doesn't exceed the maximum input size of the embedding model. That is why you usually chunk a document based on your data structure and the maximum input size of the model.
4. **Embedding:** You pass each chunk individually to an embedding model of your choice. Implementation-wise, this step is usually the simplest, as tools such as SentenceTransformer and Hugging Face provide high-level interfaces for most embedding models. As explained in the *What are embeddings?* section of this chapter, at this step, the most critical decisions are to decide what model to use and whether to fine-tune it or not. For example, we used an "all-mpnet-base-v2" embedding model from *SentenceTransformer*, which is relatively tiny and runs on most machines. However, we provide a configuration file where you can quickly configure the embedding model with something more powerful based on the state of the art when reading this book. You can quickly find other options on the MTEB on Hugging Face (<https://huggingface.co/spaces/mteb/leaderboard>).
5. **Data loading:** The final step combines the embedding of a chunked document and its metadata, such as the author and the document ID, content, URL, platform, and creation date. Ultimately, we wrap the vector and the metadata into a structure compatible with Qdrant and push it to the vector DB. As we want to use Qdrant as the single source of truth for the features, we also push the cleaned documents (before chunking) to Qdrant. We can push data without vectors, as the metadata index of Qdrant behaves like a NoSQL DB. Thus, pushing metadata without a vector attached to it is like using a standard NoSQL engine.

Change data capture: syncing the data warehouse and feature store

As highlighted a few times in this chapter, data is constantly changing, which can result in DBs, data lakes, data warehouses, and feature stores getting out of sync. **Change data capture (CDC)** is a strategy that allows you to optimally keep two or more data storage types in sync without computing and I/O overhead. It captures any CRUD operation done on the source DB and replicates it on a target DB. Optionally, you can add preprocessing steps in between the replication.

The syncing issues also apply when building a feature pipeline. One key design choice concerns how to sync the data warehouse with the feature store to have data fresh enough for your particular use case.

In our LLM Twin use case, we chose a naïve approach out of simplicity. We implemented a batch pipeline that is triggered periodically or manually. It reads all the raw data from the data warehouse, processes it in batches, and inserts new records or updates old ones from the Qdrant vector DB. This works fine when you are working with a small number of records, at the order of thousands or tens of thousands. But our naïve approach raises the following questions:

- What happens if the data suddenly grows to millions of records (or higher)?
- What happens if a record is deleted from the data warehouse? How is this reflected in the feature store?
- What if we want to process only the new or updated items from the data warehouse and not all of them?

Fortunately, the CDC pattern can solve all of these issues. When implementing CDC, you can take multiple approaches, but all of them use either a push or pull strategy:

- **Push:** The source DB is the primary driver in the push approach. It actively identifies and transmits data modifications to target systems for processing. This method ensures near-instantaneous updates at the target, but data loss can occur if target systems are inaccessible. To mitigate this, a messaging system is typically employed as a buffer.
- **Pull:** The pull method assigns a more passive role to the source DB, which only records data changes. Target systems periodically request these changes and handle updates accordingly. While this approach lightens the load on the source, it introduces a delay in data propagation. A messaging system is again essential to prevent data loss during periods of target system unavailability.

In summary, the push method is ideal for applications demanding immediate data access, whereas the pull method is better suited for large-scale data transfers where real-time updates aren't critical. With that in mind, there are different methods to detect changes in data. Thus, let's list the main CDC patterns that are used in the industry:

- **Timestamp-based:** The approach involves adding a modification time column to DB tables, usually called `LAST_MODIFIED` or `LAST_UPDATED`. Downstream systems can query this column to identify records that have been updated since their last check. While simple to implement, this method is limited to tracking changes, not deletions, and imposes performance overhead due to the need to scan entire tables.
- **Trigger-based:** The trigger-based approach utilizes DB triggers to automatically record data modifications in a separate table upon `INSERT`, `UPDATE`, or `DELETE` operations, often known as the event table. This method provides comprehensive change tracking but can impact the DB performance due to the additional write operations involved for each event.
- **Log-based:** DBs maintain transaction logs to record all data modifications, including timestamps. Primarily used for recovery, these logs can also be leveraged to propagate changes to target systems in real time. This approach minimizes the performance impact on the source DB. As a huge advantage, it avoids additional processing overhead on the source DB, captures all data changes, and requires no schema modification. But on the opposite side, it lacks standardized log formats, leading to vendor-specific implementations.



For more details on CDC, I recommend *What is Change Data Capture?* from Confluent's blog: <https://www.confluent.io/en-gb/learn/change-data-capture/>.

With these CDC techniques in mind, we could quickly implement a pull timestamp-based strategy in our RAG feature pipeline to sync the data warehouse and feature store more optimally when the data grows. Our implementation is still pull-based but doesn't check any last updated field in the source DB; it just pulls everything from the data warehouse.

However, the most popular and optimal technique in the industry is the log-based one. It doesn't add any I/O overhead to the source DB, has low latency, and supports all CRUD operations. The biggest downside is its development complexity, which requires a queue to capture all the CRUD events and a streaming pipeline to process them.

As this is an LLM book and not a data engineering one, we wanted to keep things simple, but it's important to know that these techniques exist, and you can always upgrade your current implementation when it doesn't fit your application requirements anymore.

Why is the data stored in two snapshots?

We store two snapshots of our data in the logical feature store:

- **After the data is cleaned:** For fine-tuning LLMs
- **After the documents are chunked and embedded:** For RAG

Why did we design it this way? Remember that the features should be accessed solely from the feature store for training and inference. Thus, this adds consistency to our design and makes it cleaner.

Also, storing the data cleaned specifically for our fine-tuning and embedding use case in the MongoDB data warehouse would have been an antipattern. The data from the warehouse is shared all across the company. Thus, processing it for a specific use case is not good practice. Imagine another summarization use case where we must clean and preprocess the data differently. We must create a new “Cleaned Data” table prefixed with the use case name. We have to repeat that for every new use case. Therefore, to avoid having a spaghetti data warehouse, the data from the data warehouse is generic and is modeled to specific applications only in downstream components, which, in our case, is the feature store.

Ultimately, as we mentioned in the *Core steps* section, you can leverage the metadata index of a vector DB as a NoSQL DB. Based on these factors, we decided to keep the cleaned data in Qdrant, along with the chunked and embedded versions of the documents.

As a quick reminder, when operationalizing our LLM Twin system, the create instruct dataset pipeline, explained in *Chapter 5*, will read the cleaned documents from Qdrant, process them, and save them under a versioned ZenML artifact. The training pipeline requires a dataset and not plain documents. This is a reminder that our logical feature store comprises the Qdrant vector DB for online serving and ZenML artifacts for offline training.

Orchestration

ZenML will orchestrate the batch RAG feature pipeline. Using ZenML, we can schedule it to run on a schedule, for example, every hour, or quickly manually trigger it. Another option is to trigger it after the ETL data collection pipeline finishes.

By orchestrating the feature pipeline and integrating it into ZenML (or any other orchestration tool), we can operationalize the feature pipeline with the end goal of **continuous training (CT)**.